

AI is a journey, not a destination.

The game Go, which is one of the oldest and most vexing challenges in artificial intelligence (AI), has a huge search space of about 10^{170} potential board positions, making brute-force search methods all but a thing of the past. In their groundbreaking paper, Silver et al. introduced AlphaGo, the first computer program to defeat a human professional Go player at a full-board, no-handicap game. This AI victory was not achieved by simple brute force or sheer memorization, however, but instead through the intelligent combination of deep learning and search methods that allowed AlphaGo to play at times even surprising the greatest players.

The power of AlphaGo lies in a blend of policy networks, value networks, and Monte Carlo Tree Search (MCTS). Policy networks are taught to play well, but only with fewer moves to consider. Value networks approximate win chances from any board position without having to play out full games. MCTS uses both to make selective predictions of forthcoming moves in a way that mimics intelligent thinking. This design made AlphaGo not only more cost-effective but also more "conscious" of what the most critical decisions are, almost as if having intuition.

AlphaGo's learning was in a formal pipeline. It began with supervised learning to learn a policy network (p_σ) from 30 million positions of human expert games. This 13-layer convolutional network achieved a 57% accuracy in predicting the next human move, surpassing the accuracy of any prior model. Then, this SL policy was subsequently learned by reinforcement learning to create a stronger policy network (p_p) that improved by playing against itself, not merely by learning to imitate humans but by winning. Finally, a value network (v_θ) was also learned to estimate who would win from any board state, based on data generated by self-play. This removed the need to run full games repeatedly and provided nearly correct estimates with significantly less effort.

During actual gameplay, these components were blended in the Monte Carlo Tree Search engine of AlphaGo. For every move, AlphaGo used the policy and value networks to tell the MCTS to search for good moves and steer clear of poor ones. In the simulations, each move was evaluated by summing its action value (how well it would go) and an exploration bonus (how good it

would be to do something new). The leaf nodes of the tree (where the simulation ends) were approximated with both the SL policy and the value networks so that even early moves were guided by good predictions. The result was a system that searched significantly fewer moves than older programs but made significantly more intelligent decisions.

AlphaGo's performance was brilliant. It won 494 of 495 games against top AI Go programs like Crazy Stone and Zen. Even when it didn't use tree search and performed only on its policy network trained through reinforcement, it even won 85% of games played against Pachi. The most famous result came in October 2015, when AlphaGo defeated European Go Champion Fan Hui in a 5–0 sweep, the first time a computer program had beaten a human pro without any handicap.

What struck me the most about this work and what makes it one of the most interesting AI models I've encountered in a while is how AlphaGo combines cold logic with something that looks a lot like intuition. As if the system had a sixth sense. But compared to human intuition, which is fuzzy and hard to articulate, AlphaGo's version is transparent, understandable learning steps. It first watches masters, practices by itself later, and finally learns to "feel" strong or weak positions. It's revolutionary in the sense that it suggests we can construct a kind of digital intuition by piling models based on logic and enabling them to get better through learning.

AlphaGo is a paradigm shift in what AI systems are able to learn and reason. Instead of basing it on hardcoded rules or brute-force search, it uses deep learning to build internal representations of strategy, something far more human and flexible. Its hierarchical learning process mirrors how humans learn: imitation, practice, and reflection. Its efficiency shows that brilliant design can outdo brute force. And its generalizability is promising to apply similar techniques to areas such as healthcare, finance, and robotics, where decisions must be made wisely and in a timely fashion. AlphaGo, in large part, didn't just master a game; it gave us a vision for how AI can begin to think.

Module 1 by Zhiyao Zhang introduced the fundamental concepts of machine learning and highlighted the implications of what it means for a system to learn through experience rather than explicit programming. Arguably, the most basic concept that I learned was that machine learning allows systems to improve their performance at some task by learning from data over time. The module covered the Experience, Task, and Performance (E, T, P) system that Tom Mitchell hypothesised, which helped to conceive what "learning" in the machine learning context means.

We saw three main types of tasks: regression, predicting continuous values; classification, predicting discrete classes; and clustering, an unsupervised task for clumping unlabelled data. The module also covered various learning paradigms like supervised learning (labeled data), unsupervised learning (learning patterns without labels), semi-supervised learning, and reinforcement learning. We understood the concepts of overfitting and underfitting, realizing how models can learn too much or too little from data, affecting their capacity to generalize in new situations.

With regard to the measurement of performance, we considered accuracy in the case of classification problems and mean squared error (MSE) for regression problems. We also learned about a very important concept that was introduced, namely gradient descent, the basic technique adopted in minimizing loss functions in the majority of learning algorithms. We also saw its more efficient versions, like stochastic gradient descent and mini-batch gradient descent, that are more appropriate for large datasets.

One real and very memorable example was fitting a linear model to three data points. A trivial example was employed to illustrate how computational modeling by way of mathematics and gradient descent optimization in reality works. It also illustrated the importance of computational

efficiency and the limitations that exist when trying to compute exact solutions to practical problems.

Then, Module 2 by Qinhang (William) Wu provided us with an in-depth overview of modern Transformer models and how they are used in numerous domains. We understood how Transformers work behind the scenes, i.e., the role of the attention mechanisms, and why these models can generalize across forms of data, e.g., text, images, and speech. One of the most significant insights was that Transformers can work with data from various domains since they convert all the inputs into a common representation, representing them as fixed-size embeddings, irrespective of whether the input is text, an image, or audio.

We explored real-world applications of Transformers in natural language processing tasks such as text classification, summarization, and translation. In computer vision, we saw how Transformers are being used for image classification and object detection, specifically by Vision Transformers (ViTs). Model flexibility was also seen in applications to speech recognition and time-series prediction.

One of the most important concepts was the attention mechanism, which enables the model to catch important relationships between elements of the input. We also learned multi-head attention, enabling the model to attend to multiple types of relationships at the same time, and positional encoding, which gives the model an idea of order in sequence. The building blocks we learned were tokenization and embedding techniques, encoder blocks composed of attention and feed-forward networks, and classification heads for task-specific outputs.

One of the most fascinating topics was the Vision Transformer (ViT), as it processes images by splitting them into patches and processing them in a similar way as words are processed in text. The architectural innovation demonstrated how applying the same model architecture to new areas of data is potent. During lab hours, we performed sequence classification practices with

transformer encoders and converted Transformers to vision tasks using open-source libraries like PyTorch and Hugging Face. These hands-on practices provided us with actual-life experience of the way these models are trained and used in reality.

During this REU program, I gained complete knowledge of networking, wireless edge technologies, and AI applications in networking and generative modeling. In Week 3, I learned the fundamentals of computer networking and wireless network architecture, including network core and edge, packet switching, and 5G RAN components like user equipment (UE), gNodeB, and control and user plane separation (CUPS), which enhanced my knowledge of mobility, latency, energy efficiency, and flexibility introduced by software-defined RAN. In Week 4, I was taught about Generative Adversarial Networks (GANs) and diffusion models, understanding how GANs generate realistic samples through an adversarial process but face problems like mode collapse, while diffusion models produce high-quality outputs through iterative denoising with slower computation, observing trade-offs and applications in content creation and medical imaging. By Week 5, I had discovered how "software-i-zation" and Software-Defined Networking (SDN) transform 5G and beyond into microservice-like architectures, with main challenges being mobility and handover, beam steering, and RAN packet scheduling, where AI/ML can be leveraged to optimize network performance and QoS through predictive modeling. Overall, this REU experience enhanced my technical expertise and interdisciplinary insight, putting me in a position to carry out future research at the intersection of wireless networking and machine learning.

During this REU program, I studied extensively about the latest concepts in networking, wireless edge technologies, and cutting-edge AI applications in networking and generative modeling. During Week 3, I was taught the fundamental elements of computer networking and wireless system architecture. These were the division of the network core and edge, the packet switching function, and the wireless network topology consisting of user devices, base stations, and wireless links. I explored the 5G RAN, its components like user equipment (UE) and gNodeB, and the separation of control and user planes (CUPS). This lesson made me understand how

mobile and IoT devices communicate in different scenarios, why latency is important, mobility is necessary, and energy efficiency matters, and how software-defined RAN provides flexibility in network management.

In Week 4, I learned about Generative Adversarial Networks (GANs) and diffusion models, broadening the machine learning horizon from classification and regression to generative tasks. I found out that GANs employ a generator-discriminator adversarial process for producing realistic samples, and the issues that arise, such as mode collapse and vanishing gradients. Then came diffusion models, in which high-quality output is produced by iteratively stripping noise through a forward and reverse process. Comparing these methods enabled me to appreciate the trade-offs: GANs provide efficient sampling but unstable training, while diffusion models give stable high-fidelity output but are slower computationally. This seminar enhanced my appreciation for the mathematical underpinnings of generative models, their uses in medical imaging and content generation, and the value of assessing generative quality beyond likelihood scores.

By Week 5, the seminar had intersected AI with advanced wireless networking, such as 5G and beyond. I learned how the "software-i-zation" of networks reorganizes historically hardware-focused operations into a nimble, microservice-like design made possible by Software-Defined Networking (SDN). We discussed three principal challenges of 5G networks: mobility and handover, beam assignment and steering, and packet scheduling at the RAN—and how AI/ML can enhance performance through predicting user behavior, antenna setup optimization, and quality-of-service (QoS) guarantees. This session allowed me to connect high-level machine learning to real networking problems, demonstrating how predictive models can drive intelligent network management at the edge.

Together, these seminars taught me about how next-generation wireless networks are emerging and how AI is transforming network operations as well as data generation. I did not only learn technical information about 5G structure, wireless communications, and generative models, but also the interdisciplinary context of next-generation AI-based networking. What I have learned will guide my future research on networked systems and machine learning applications.

work cited

Silver, David, et al. "*Mastering the Game of Go with Deep Neural Networks and Tree Search.*"

Nature, vol. 529, no. 7587, 2016, pp. 484–489. <https://doi.org/10.1038/nature16961>.